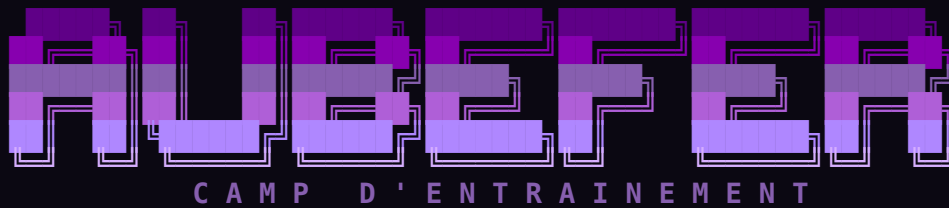


PROJET RED · YNOV CAMPUS AIX · B1 INFORMATIQUE



Un jeu de rôle en terminal, écrit en Go, sans une seule dépendance.

Chargement du camp [#####]

---

Théo SUGIER · Amine KOUAIDI · Martim GALHARDO

soutenance · 25.09.2026

## LE CONCEPT

# Vous êtes une recrue. Le Sergent vous attend.

Le sujet impose des fonctions, pas un univers. On a choisi le décor d'abord : un camp d'entraînement, où chaque fonction demandée trouve une raison d'exister.

### Un lieu, pas un menu

L'inventaire est une **besace**, le marchand un **Régisseur**, le combat un **terrain d'exercice**.

### Une voix qui guide

Le Sergent commente chaque action, lettre par lettre. Aucun message technique n'atteint jamais l'écran.

### Go seul, rien d'autre

Bibliothèque standard uniquement. Le correcteur lance le jeu sans rien installer.

# Le parcours du joueur

01

## Écran titre

La bannière se dessine,  
la barre de chargement  
se remplit, le Sergent  
vous hèle.



02

## Engagement

Le nom est validé  
caractère par caractère,  
puis on choisit son  
lignage.



03

## Le camp

Fiche, besace,  
Régisseur, terrain  
d'exercice, palissade.



04

## Le combat

Tour par tour contre le  
gobelin : coups, potions,  
sorts.

On revient toujours au camp après un combat. Il n'y a pas de cul-de-sac dans le jeu.

# Le camp, tel qu'il s'affiche

go run main.go

– écran titre –

AUBEFER  
CAMP D'ENTRAINEMENT

Chargement du camp [#####]  
« Encore une recrue. Voyons ce que tu vaux. »  
Appuie sur Entree pour entrer au camp...

– le camp –

)  
( ) (  
) ( )  
-( )-  
'-----'

LE CAMP D'AUBEFER

Le feu crepite. Le Sergent t'attend.

-----  
[1] Ta fiche de recrue  
[2] Ta besace  
[3] La tente du Regisseur  
[4] Le terrain d'exercice  
[0] Quitter le camp  
-----

Ton choix :

Le feu de camp est la seule chose chaude de la palette. Tout le reste du jeu est en dégradé violet, du plus sombre au plus clair.



## Ce qu'on a livré

0/22

tâches écrites

Les trois parties  
du sujet sont  
couvertes.

0

missions  
bonus

Combat magique,  
mana, habillage  
ASCII.

0

lignes de Go

14 fichiers, un par  
domaine du jeu.

0

commits sur

`main`

Une branche et  
une relecture par  
tâche.

0

dépendance  
externe

Rien à installer :

`go run main.go`

Trois finitions qu'on assume plutôt que de les cacher : la Forge et l'équipement sont écrits mais pas encore reliés au menu, et l'initiative vit dans les structures sans piloter l'ordre des tours.

## L'ARCHITECTURE

# Un point d'entrée, un fichier par domaine

main.go

```
package main

// le jeu vit dans le dossier src, ici je ne fais que le lancer
import "aubefer/src"

func main() {
    src.Start()
}
```

**Start** prend une majuscule parce qu'en Go un nom capitalisé est visible hors du package. C'est la seule porte d'entrée de `src`.

```
src/
game.go      le joueur, l'écran titre
creation.go  nom, lignage
menu.go      le camp
inventory.go la besace
merchant.go  le Régisseur
blacksmith.go la Forge
combat.go    le terrain
ascii.go     l'habillage
```

### Pourquoi découper comme ça

Ce n'est pas esthétique, c'est du Git. Quand Amine travaille sur `merchant.go` et Martim sur `monster.go`, il n'y a pas de conflit.

DÉMONSTRATION

# On vous montre le jeu.

THÉO

Lancer, créer la recrue, la  
fiche, la besace

AMINE

Le Régisseur, l'or, les prix

MARTIM

Le combat : coup, potion,  
sort, victoire

THÉO

Retour au camp, quitter  
proprement

Trois minutes. Le scénario est répété : on sait où on va, on ne cherche rien devant vous.

# Comment on s'est organisés

## Trello — une carte par tâche

28 cartes, une couleur par personne. Une carte passe en Terminé quand un autre membre a relu, pas avant.

## Git — une branche par tâche

Personne ne pousse sur `main`. Une branche `feat/t22-training-fight`, une Pull Request, une relecture, puis la fusion.

## Des conventions écrites

Commits en `type(portée) : description`. Le code et les noms de fonctions en anglais, tout le texte joueur en français.

## La règle qui a tout tenu

**On relance le jeu avant chaque push.** Si `main` est cassé, les deux autres sont bloqués.

# Qui a fait quoi

## Théo

NOYAU · INTERFACE · INTÉGRATION

- Menu du camp et boucle principale
- Création du personnage, validation du nom
- Combat : dégâts, tour du joueur, tour du goblin
- Habillage ASCII, couleurs, barres de vie
- Dépôt, conventions, résolution des conflits

## Amine

ÉCONOMIE · MAGIE

- La besace : ajouter, retirer, limite de place
- Le Régisseur, l'or et les prix
- La Forge et les recettes
- Le livre de sorts et le mana
- Équiper un objet, recalcul des PV
- Potions de soin et de poison

## Martim

PERSONNAGE · MONSTRE

- La structure `Character` et son initialisation
- L'affichage de la fiche
- La structure `Equipment`
- Le goblin : `Monster` et `initGoblin`

RETOUR D'EXPÉRIENCE

# Les quatre murs qu'on s'est pris

| Le problème                                            | Ce qu'on a fait                                                                                                                 |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Le jeu ne se lançait pas chez deux d'entre nous        | <code>go.mod</code> exigeait une version de Go plus récente que la leur. On l'a fixée à une version que les trois machines ont. |
| Des conflits Git à chaque fusion                       | On touchait tous au même fichier. On a découpé en un fichier par domaine : les conflits ont quasiment disparu.                  |
| Le menu répondait tout seul                            | Deux façons de lire le clavier coexistaient et se marchaient dessus. Règle adoptée : un seul lecteur d'entrée dans tout le jeu. |
| Des marqueurs de conflit poussés sur <code>main</code> | <code>main</code> ne compilait plus pour personne. D'où la règle : on relance le jeu avant chaque push.                         |

À chaque fois, la correction est devenue une règle d'équipe. C'est ça qu'on retient, plus que le bug lui-même.

# Objectifs atteints, et ce qu'on en retire

## Les objectifs

**Les 22 tâches du sujet** — livrées et jouables.

**Un jeu présentable** — aucun message technique à l'écran, tout est dans l'univers.

**Un dépôt propre** — branches, Pull Requests, documentation versionnée.

**Trois bonus** — sorts, mana, habillage ASCII.

## Ce qu'on a vraiment appris

**Go** — pointeurs, slices, structures, méthodes, et pourquoi une majuscule change tout.

**Git à plusieurs** — un conflit n'est pas une panne, c'est une décision à prendre.

**Se parler tôt** — une structure partagée se décide à trois, pas dans son coin.

**Finir avant d'enjoliver** — le socle d'abord, les bonus ensuite.

# Ce qu'on ferait autrement

## À REFAIRE

Figurer les structures le premier jour

`Character` et `Monster` ont bougé trop longtemps. Une demi-journée à trois au départ nous aurait évité plusieurs soirées de fusion.

## À AJOUTER

Des tests automatiques

On a tout testé à la main. Quelques tests sur la validation du nom et sur la besace auraient attrapé deux bugs avant nous.

## LA SUITE

Finir l'initiative, puis l'expérience

L'ordre des tours selon le lignage est à brancher, et la montée de niveau viendrait juste après.

# Merci.

On répond à vos questions — et on peut relancer le jeu sur n'importe quel passage.

---

Théo SUGIER · Amine KOUAIDI · Martin GALHARDO

[github.com/YujinW3b/ProjetRed\\_Theo\\_Amine\\_Martin](https://github.com/YujinW3b/ProjetRed_Theo_Amine_Martin)

Deux slides de code suivent, pour le temps d'échange. **0** ouvre le sommaire.

## ANNEXE

# Valider un nom, caractère par caractère

src/creation.go

```
func isValidName(name string) bool {
    if len(name) == 0 {
        return false
    }

    for i := 0; i < len(name); i++ {
        c := name[i]

        minuscule := c >= 'a' && c <= 'z'
        majuscule := c >= 'A' && c <= 'Z'

        if !(minuscule || majuscule) {
            return false
        }
    }
    return true
}
```

## Pourquoi comparer des caractères

Un caractère est un nombre. `'a'` vaut 97, `'z'` vaut 122 : tout ce qui tombe entre les deux est une minuscule. Pas besoin de bibliothèque.

## Le piège des accents

`name[i]` renvoie un **octet**, pas une lettre. Un « é » occupe deux octets, donc il est refusé — d'où le message du Sergent : « *Des lettres, rien d'autre. Pas d'accent.* »

## Et juste après

`capitalize` copie la chaîne dans un `[]rune`, parce qu'une string ne se modifie pas en Go, et décale l'initiale de 32 dans la table ASCII.

## ANNEXE

# Pourquoi le goblin passe par son adresse

src/combat.go

```
func characterTurn(goblin *Monster) bool {  
    action := askCombatAction()  
  
    switch action {  
    case "attack":  
        goblin.Pv = goblin.Pv - 5  
        if goblin.Pv < 0 { goblin.Pv = 0 }  
        return true  
    case "inventory":  
        takePot(&joueur)  
        return true  
    case "spell":  
        return castSpell(goblin)  
    }  
    return false  
}
```

## Sans l'étoile, les dégâts disparaissent

Go passe les valeurs par copie. Avec `goblin` `Monster`, la fonction taperait sur une photocopie : les 5 points partiraient à la poubelle en sortant. `*Monster` donne l'adresse du vrai goblin.

## Pourquoi un booléen en retour

Il dit si le tour a été **consommé**. Un sort annulé faute de mana rend la main au joueur au lieu de lui faire perdre son tour.

## Le même piège côté joueur

`goblinPattern` renvoie aussi un booléen, parce que `isDead` remet le joueur debout : tester `Pv <= 0` après coup ne détecterait jamais la mort.